```
A simple grayscale image can be represented as a 2D NumPy array where each number represents pixel intensity.

image = np.array([
    [10, 20, 30, 40, 50],
    [20, 30, 40, 50, 60],
    [30, 40, 50, 60, 70],
    [40, 50, 60, 70, 80],
    [50, 60, 70, 80, 90]
])
Tasks
Find the dimensions of the image.
Find its shape.
Find the total number of pixels.
Display the center pixel.
Extract the top-left 3 × 3 region.
Extract the bottom-right 2 × 2 region.
Increase the intensity of the center 3 × 3 region by 20.
Display the modified image.
```

```
import numpy as np

image = np.array([
    [10, 20, 30, 40, 50],
    [20, 30, 40, 50, 60],
    [30, 40, 50, 60, 70],
    [40, 50, 60, 70, 80],
    [50, 60, 70, 80, 90]
])
```

```
image.ndim
```

```
2
```

```
image.shape
```

```
(5, 5)
```

```
image.size
```

```
25
```

```
image[2,2]
```

```
np.int64(50)
```

```
image[:3, :3]
```

```
array([[10, 20, 30],
       [20, 30, 40],
       [30, 40, 50]])
```

```
image[-2:,-2:]
```

```
array([[70, 80],
       [80, 90]])
```

```
image
```

```
array([[10, 20, 30, 40, 50],
       [20, 30, 40, 50, 60],
       [30, 40, 50, 60, 70],
       [40, 50, 60, 70, 80],
       [50, 60, 70, 80, 90]])
```

```
# Increase the intensity of the center 3 × 3 region by 20.
image[1:4,1:4] += 20
```

```
image[1:4,1:4]
```

```
array([[50, 60, 70],
       [60, 70, 80],
```

```
       [70, 80, 90]])
```

```
print(image)
```

```
[[10 20 30 40 50]
 [20 50 60 70 60]
 [30 60 70 80 70]
 [40 70 80 90 80]
 [50 60 70 80 90]]
```

```
A company stores quarterly revenue and expenses:

financials = np.array([
    [120, 80],
    [150, 90],
    [180, 110],
    [200, 130]
])

Rows = Q1, Q2, Q3, Q4
Columns = Revenue, Expense

Tasks
Display Q2's complete financial data.
Display all quarterly revenues.
Display all quarterly expenses.
Extract Q2 and Q3 data.
Increase Q4 revenue by 15%.
Reduce Q1 expense by 10%.
Display the final financial matrix.
```

```
financials = np.array([
    [120, 80],
    [150, 90],
    [180, 110],
    [200, 130]
])
```

```
# Display Q2's complete financial data.

financials[1]
```

```
array([150,  90])
```

```
# Display all quarterly revenues.
financials[:,0]
```

```
array([120, 150, 180, 200])
```

```
# Display all quarterly expenses.
financials[:,1]
```

```
array([ 80,  90, 110, 130])
```

```
financials[:,-1]
```

```
array([ 80,  90, 110, 130])
```

```
# Extract Q2 and Q3 data.
financials[1:3,]
```

```
array([[150,  90],
       [180, 110]])
```

```
# Increase Q4 revenue by 15%.
financials[3,0] = financials[3,0]*1.15
print(financials)
```

```
[[120  80]
 [150  90]
 [180 110]
 [229 130]]
```

```
# Reduce Q1 expense by 10%.
financials[0,1] = financials[0,1]*0.9
```

```
print(financials)
```

```
[[120  72]
 [150  90]
 [180 110]
 [229 130]]
```

```
A classroom has 6 rows and 5 seats per row.

Initially, all seats are empty.

0 = Empty
1 = Occupied
Tasks
Create a 6 × 5 array filled with zeros.
Mark the first seat of the first row as occupied.
Mark all seats in the 3rd row as occupied.
Mark the last two seats of the last row as occupied.
Display the first 3 rows.
Display the last 2 columns.
Display the complete seating matrix.
```

```
# Create a 6 × 5 array filled with zeros.
seat = np.full((6,5), 0)
seat
```

```
array([[0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0]])
```

```
# Mark the first seat of the first row as occupied.
seat[0,0] = 1
```

```
seat
```

```
array([[1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0]])
```

```
# Mark all seats in the 3rd row as occupied.
seat[2] = 1
```

```
seat
```

```
array([[1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0]])
```

```
# Mark the last two seats of the last row as occupied.
seat[5,-2:] = 1
```

```
seat
```

```
array([[1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 1, 1]])
```

```

```

```
# Display the first 3 rows.
seat[:3]
```

```
array([[1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1]])
```

```
# Display the last 2 columns.
seat[:,-2:]
```

```
array([[0, 0],
       [0, 0],
       [1, 1],
       [0, 0],
       [0, 0],
       [1, 1]])
```

```
seat
```

```
array([[1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 1, 1]])
```

```
sales = np.array([100, 200, 300, 400, 500])
sales
```

```
array([100, 200, 300, 400, 500])
```

```
sm = 0
for elem in sales:
  sm += elem
print(sm)
```

```
1500
```

```
sm = 0
n = len(sales)
for elem in sales:
  sm += elem
print(sm/n)
```

```
300.0
```

```
np.sum(sales)
```

```
np.int64(1500)
```

```
np.avg(sales)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_1548/3212801408.py in <cell line: 0>()
----> 1 np.avg(sales)

/usr/local/lib/python3.13/dist-packages/numpy/__init__.py in __getattr__(attr)
    412             return char.chararray
    413
--> 414         raise AttributeError("module {!r} has no attribute "
    415                              "{!r}".format(__name__, attr))
    416

AttributeError: module 'numpy' has no attribute 'avg'
```

Next steps: Explain error

```
np.mean(sales)
```

```
np.float64(300.0)
```

```
#fine the min salary

sales = np.array([100, 200, 300, 400, 500])

mn = 0

for s in sales:
  if s < mn:
    mn = s

print(mn)
```

```
0
```

```
#fine the min salary

sales = np.array([100, 200, 300, 400, 500])

mn = sales[0]

for s in sales:
  if s < mn:
    mn = s

print(mn)
```

```
100
```

```
#fine the max salary

sales = np.array([100, 200, 300, 400, 500])


mx = 0

for s in sales:
  if s > mx:
    mx = s

print(mx)
```

```
500
```

```
#fine the max salary

sales = np.array([-100, -200, -300, -400, -500])


mx = 0

for s in sales:
  if s > mx:
    mx = s

print(mx)
```

```
0
```

```
#fine the max salary

sales = np.array([-100, -200, -300, -400, -500])


mx = sales[0]

for s in sales:
  if s > mx:
    mx = s

print(mx)
```

```
-100
```

```
# #fine the max salary
```

```
# sales = np.array([-100, -200])


# mx = sales[2]

# for s in sales:
#   if s > mx:
#     mx = s

# print(mx)
```

```
sales = np.array([100, 200, 300, 400, 500])


np.min(sales)
```

```
np.int64(100)
```

```
np.max(sales)
```

```
np.int64(500)
```

```
sales = np.array([50, 30, 10, 190, 100 ])

# median?
```

```
A teacher has recorded marks of 10 students:

marks = np.array([72, 85, 91, 64, 78, 88, 95, 70, 82, 76])

Find:

Total marks
Average marks
Highest marks
Lowest marks
Difference between highest and lowest marks
Index of the student with the highest marks
```

```
marks = np.array([72, 85, 91, 64, 78, 88, 95, 70, 82, 76])

total = np.sum(marks)
total
```

```
np.int64(801)
```

```
avg = np.mean(marks)
avg
```

```
np.float64(80.1)
```

```
np.min(marks)
```

```
np.int64(64)
```

```
np.max(marks)
```

```
np.int64(95)
```

```
np.max(marks) - np.min(marks)
```

```
np.int64(31)
```

```
marks
```

```
array([72, 85, 91, 64, 78, 88, 95, 70, 82, 76])
```

```
mx = marks[0]
mxi = 0
n = len(marks)

for i in range(n):
```

```
  if mx < marks[i]:
    mx = marks[i]
    mxi = i

print(mx)
print(mxi)
```

```
95
6
```

```
np.argmax(marks)
```

```
np.int64(6)
```

```
marks = np.array([72, 85, 91, 64, 78, 88, 95, 70, 95, 82, 76])
np.argmax(marks)
```

```
np.int64(6)
```

A company's daily sales for 7 days are:

sales = np.array([45000, 47000, 49000, 52000, 50000, 48000, 250000])

Calculate the mean and median.

Then determine which measure better represents a typical sales day when considering the unusually large sale.

Also calculate the standard deviation.

```
sales = np.array([45000, 47000, 49000, 52000, 50000, 48000, 250000])
```

```
np.mean(sales)
```

```
np.float64(77285.71428571429)
```

```
np.median(sales)
```

```
np.float64(49000.0)
```

```
np.std(sales)
```

```
np.float64(70540.18973143565)
```

A business records revenue for 12 months:

```
revenue = np.array([
    120000, 135000, 128000, 145000,
    150000, 138000, 160000, 172000,
    155000, 148000, 180000, 195000
])
```

Find:

Average monthly revenue
Highest revenue
Lowest revenue
Month index with highest revenue
Month index with lowest revenue
Total annual revenue
Difference between highest and lowest revenue

The following array contains sales made by 4 salespeople across 3 months.

```
sales = np.array([
    [120, 150, 180],
    [200, 170, 160],
    [100, 140, 130],
    [180, 190, 210]
])
```

Rows represent salespeople and columns represent months.

```
Find:

Total sales by each salesperson
Average sales by each salesperson
Total sales for each month
Average sales for each month
Highest sales achieved by each salesperson
```